

Técnicas Digitales 2 - Trabajo práctico N° 2 - Act N°3

Braida Agustín (404621) - Ernst Pedro (400624) - Polizze Tomas (400793) - Valentino Rao (402308)

**Técnicas Digitales 2 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional**

I. INTRODUCCIÓN

Este ejercicio documenta la transición tecnológica del firmware para la placa BluePill desde un enfoque de programación *Bare Metal* manual hacia el estándar CMSIS (Cortex Microcontroller Software Interface Standard). El objetivo principal es abandonar el acceso directo a memoria mediante punteros manuales y adoptar las estructuras de datos provistas por el fabricante (stm32f10x.h), mejorando la legibilidad y el mantenimiento del firmware.

II. INSTALACIÓN Y JUSTIFICACIÓN DEL ENTORNO CMSIS

La migración a CMSIS requiere la integración de archivos específicos, donde cada uno cumple un rol crítico para la estabilidad y portabilidad del sistema.

III. MIGRACIÓN: DEL ACCESO MANUAL A ESTRUCTURAS CMSIS

En el enfoque BareMetal puro, el acceso se realizaba mediante macros que apuntaban a direcciones específicas:

```
1 #define RCC_APB2ENR *(volatile uint32_t *)
   (0x40021018)
2 RCC_APB2ENR |= (1 << 2); // Habilitar IOPA
```

Listing 1. Enfoque BareMetal anterior

Con CMSIS, se utilizan estructuras que agrupan los registros de cada periférico, lo que permite un acceso más semántico:

```
1 RCC->APB2ENR |= RCC_APB2ENR_IOPAEN; // Uso
   de estructuras y mascaras predefinidas
```

Listing 2. Enfoque CMSIS

Esta transición elimina la necesidad de calcular desplazamientos manuales (offsets) y reduce errores humanos al utilizar nombres simbólicos para los bits.

IV. IMPLEMENTACIÓN DE LA BASE DE TIEMPO (SysTick)

Se configuró el **SysTick** para generar una interrupción cada **1 ms**. En el código implementado, esto permite gestionar el parpadeo del LED integrado (PC13) de forma asíncrona mediante la variable `tick_led`, sin bloquear el flujo principal del programa.

```
*****
```

V. COMPARATIVA DE ESTRUCTURA: BARE METAL VS. CMSIS

Una de las diferencias más notables en esta migración es la reorganización del espacio de trabajo. El proyecto ha pasado de una estructura plana con archivos manuales a una organización jerárquica basada en el estándar industrial.

A. Evolución del Árbol de Directorios

A continuación se comparan las dos estructuras de trabajo:

Actividad 2 (Bare Metal)

- `main.c` (Acceso manual)
- `crt.s` (Startup manual)
- `linker.ld` (Script genérico)
- `Makefile` (Simple)

Actividad 3 (CMSIS)

- `src/main.c`
- `CMSIS/` (Librerías ARM/ST)
- `ld/STM32F103XB_FLASH.ld`
- `Makefile` (Configurado para CMSIS)

B. Sustitución de Archivos Críticos

Siguiendo la guía de instalación, se realizaron los siguientes reemplazos fundamentales para garantizar la compatibilidad con el estándar:

- **De `crt.s` a `startup_stm32f103xb.s`:** El antiguo archivo `crt.s` era una implementación mínima de la tabla de vectores. El nuevo archivo de ST no solo incluye los vectores de interrupción, sino que está estandarizado para trabajar con el `Reset_Handler` de CMSIS y permite una gestión de excepciones mucho más robusta [3].
- **De `linker.ld` a `STM32F103XB_FLASH.ld`:** Se abandonó el script de enlazado genérico por uno provisto por el fabricante. Este nuevo archivo define con precisión las regiones de memoria FLASH y RAM de la BluePill, además de gestionar correctamente las secciones `.text`, `.data` y los tamaños mínimos de `stack` y `heap` necesarios para las librerías de C [4, 8].
- **Incorporación de `system_stm32f1xx.c`:** Este archivo no existía en la versión Bare Metal pura. Su función

es crítica ya que implementa la inicialización de los relojes (clocks), permitiendo que el sistema arranque directamente a **72 MHz** mediante la función `SystemInit()`, lo cual simplifica enormemente el código en el `main.c` [3].

- **Inclusión de `syscalls.c`:** Debido a que el nuevo startup convoca funciones de la librería `newlib`, este archivo es necesario para proveer las implementaciones mínimas de entrada/salida y evitar errores de enlazado [8].

C. El Directorio CMSIS/

El nuevo directorio `CMSIS/` centraliza tanto el núcleo (Core) de ARM como el soporte de dispositivo (Device) de ST. Esto permite que el proyecto sea escalable: si quisiéramos cambiar a otro microcontrolador de la misma familia, solo tendríamos que actualizar los archivos dentro de esta carpeta, manteniendo la lógica de nuestra aplicación en `src/` prácticamente intacta.

VI. COMPARATIVA DE ESTRUCTURA DE PROYECTO: BARE METAL VS. CMSIS

La migración al estándar CMSIS no solo modifica la sintaxis del código, sino que exige una reorganización completa del árbol de directorios para integrar las librerías del fabricante y del núcleo ARM de manera ordenada [2].

A. Evolución del Directorio de Trabajo

Mientras que en la Actividad 2 se utilizaba una estructura simple con archivos manuales en la raíz del proyecto, la Actividad 3 adopta una estructura compartimentada para mejorar la escalabilidad y el mantenimiento [2, 3].

Estructura Actividad 2 (Manual)

- `main.c` (Código aplicación)
- `crt.s` (Startup manual)
- `linker.ld` (Script genérico)
- `Makefile` (Básico)

Estructura Actividad 3 (CMSIS)

- **CMSIS/:** Librerías ARM y ST
- **src/:** `main.c` y `syscalls.c`
- **ld/:** `STM32F103XB_FLASH.ld`
- **Makefile:** Configurado para CMSIS

VII. CÓDIGO FINAL: MAIN.C

A continuación, se presenta el código fuente utilizado para el ejercicio, el cual integra las interrupciones externas (EXTI), la lógica del ADC y la base de tiempo SysTick bajo el estándar CMSIS.

```

1  #include "stm32f1xx.h"
2
3  // --- Variables Globales ---
4  volatile uint8_t modo_adc = 0;
5  volatile uint32_t tick_led = 0;
6
7  void delay_dummy(volatile uint32_t limite)
8  {
9      for (volatile uint32_t i = 0; i <
10         limite; i++);
11 }
12
13 // =====
14 // RUTINAS DE SERVICIO DE INTERRUPCION (
15 //   ISRs)
16 // =====
17
18 // Parpadeo asincrono del LED integrado (
19 //   PC13) via SysTick
20 void SysTick_Handler(void) {
21     tick_led++;
22     if (tick_led >= 500) {
23         GPIOC->ODR ^= (1 << 13);
24         tick_led = 0;
25     }
26 }
27
28 // Boton Debil en PA0 (EXTI0) -> Prioridad
29 //   BAJA
30 void EXTI0_IRQHandler(void) {
31     modo_adc = !modo_adc; // Alterna la
32     // fuente del ADC
33 }
34
35 // Boton Fuerte en PA1 (EXTI1) -> Prioridad
36 //   ALTA
37 void EXTI1_IRQHandler(void) {
38     GPIOA->ODR |= (1 << 3);
39     delay_dummy(1500000);
40     GPIOA->ODR &= ~(1 << 3);
41 }
42
43 // =====
44 // MAIN PROGRAM
45 // =====
46
47 int main(void) {
48     // 1. Clocks: Habilitar ADC1, GPIOC,
49     //   GPIOB, GPIOA y AFIO usando macros
50     //   CMSIS
51     RCC->APB2ENR |= RCC_APB2ENR_ADC1EN |
52     RCC_APB2ENR_IOPCEN |
53     RCC_APB2ENR_IOPBEN |
54     RCC_APB2ENR_IOPAEN |
55     RCC_APB2ENR_AFIOEN;
56
57     // Configuración de perifericos y bucle
58     //   principal...
59 }

```

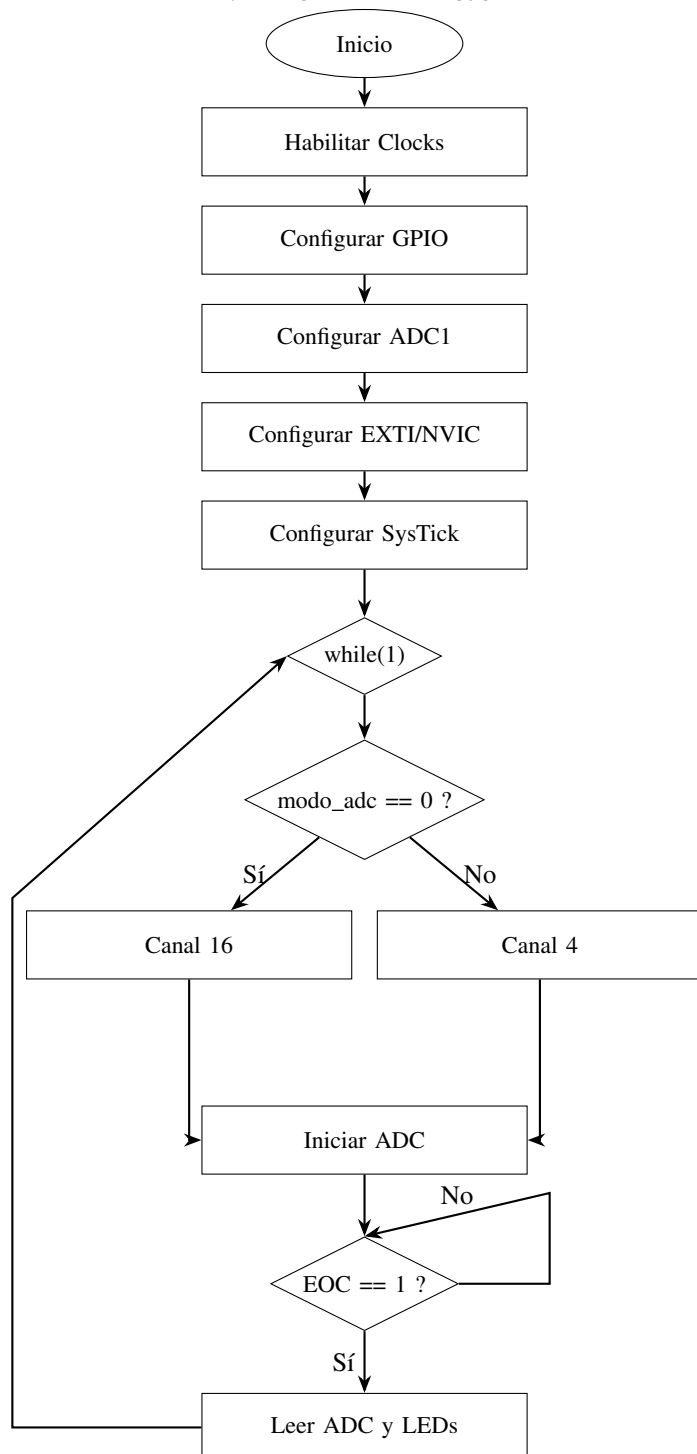
Listing 3. Código fuente `main.c` con CMSIS y SysTick

VIII. ANÁLISIS COMPARATIVO Y CONCLUSIÓN

La migración a CMSIS ha demostrado ser superior en términos de ****mantenimiento**** y ****organización****. Al utilizar el estándar de ARM, el código es más fácil de depurar ya que los nombres de los registros e interrupciones coinciden exactamente con la documentación técnica del fabricante.

La mayor dificultad encontrada fue la correcta configuración de los archivos de *startup* y el *linker script*, pero el aprendizaje sobre la gestión de interrupciones asíncronas y la precisión temporal con SysTick es fundamental para el desarrollo de sistemas embebidos de tiempo real.

IX. DIAGRAMA DE FLUJO



REFERENCIAS

- [1] DATA-SHEET STM32F103
- [2] Archivo PDF del TP2 Técnicas Digitales 2